SETUP AND ACCESS

Control MCP server access for your organization

Restrict which MCP servers users can add or connect to with managed configuration files, allowlists, and denylists.

By default, anyone running Claude Code can connect any **MCP server** they choose. Anthropic reviews connectors against its **listing criteria** before adding them to the **Anthropic Directory**, but doesn't security-audit or manage any MCP server. As an administrator, you can restrict which servers run in your organization, from deploying a fixed approved set to disabling MCP entirely.

This page covers how to:

- **Choose a pattern** that matches how much control you need
- **Deploy a fixed server set with** `managed-mcp.json`, including how to **disable MCP entirely**
- **Control servers with allowlists and denylists**
- **Tell users what to expect** when a restriction blocks a server
- **Monitor which servers your organization actually uses**

> The **Security** page covers the MCP threat model and how to evaluate a server before approving it. **Decide what to enforce** covers MCP restrictions alongside the other administrative controls.

Choose a pattern

Claude Code supports a range of restriction levels. Each pattern uses one or both of the mechanisms covered below: `managed-mcp.json` for deploying a fixed set, and `allowedMcpServers` / `deniedMcpServers` for filtering what users configure.

Pattern	What it does	Configure
Disable MCP	No servers load anywhere	`managed-mcp.json` with an empty server map
Fixed deployment	Every user gets the same servers and can't add others	`managed-mcp.json` with the servers you want

Pattern	What it does	Configure
Approved catalog	Publish a list of approved servers; users add the ones they want, anything else is blocked	`allowedMcpServers` + `allowManagedMcpServersOnly: true`
Plugin servers only	Servers can only come from plugins; users can't add their own	`strictPluginOnlyCustomization` with `mcp` in the list
Soft allowlist	Enforce an allowlist that users can broaden in their own settings	`allowedMcpServers` without `allowManagedMcpServersOnly`
Denylist only	Block known-bad servers, allow everything else	`deniedMcpServers`
No restrictions	Users add anything	Don't deploy any managed MCP configuration

Claude Code doesn't have a built-in MCP server registry that users can browse and install from. For the approved-catalog pattern, share the approved list and its `claude mcp add` commands somewhere your users will find them, such as an internal wiki, or distribute the servers as plugins through a **managed plugin marketplace** so users can browse and install them from `/plugin`.

Exclusive control with managed-mcp.json

If you deploy a `managed-mcp.json` file, Claude Code loads only the servers that file defines. Users cannot add, modify, or use any other MCP servers, including plugin-provided servers. The file also suppresses claude.ai connectors unless you **allow them alongside the managed set**.

Two other settings can further filter the managed set:

- `allowedMcpServers` and `deniedMcpServers` apply to managed servers too, so a managed server that doesn't pass them won't load.
- A user's own `deniedMcpServers` merges in from their settings, so users can block a managed server for themselves.

See **How a server is evaluated** for the full order of checks.

`managed-mcp.json` is a standalone file, so it cannot be delivered through **server-managed settings**. Any process that can write to a system path with administrator privileges can deploy it. At scale, that's usually through device management tooling, such as Jamf or a configuration profile on macOS, Group Policy or Intune on Windows, or your fleet management of choice on Linux. Claude Code looks for the file at one of these paths:

Platform	Path
macOS	`/Library/Application Support/ClaudeCode/managed-mcp.json`
Linux and WSL	`/etc/claude-code/managed-mcp.json`
Windows	`C:\Program Files\ClaudeCode\managed-mcp.json`

The file uses the same format as a project `.mcp.json` file:

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "sentry": {
      "type": "http",
      "url": "https://mcp.sentry.dev/mcp"
    },
    "company-internal": {
      "type": "stdio",
      "command": "/usr/local/bin/company-mcp-server",
      "args": ["--config", "/etc/company/mcp-config.json"],
      "env": {
        "COMPANY_API_URL": "https://internal.example.com"
      }
    }
  }
}
```

Authenticate with per-user credentials

Any user on the machine can read this file, so don't store API keys or other credentials in `env` blocks. Pass per-user credentials with one of these instead:

- **`${VAR}` expansion** to read secrets from each user's environment.
- **OAuth or per-user headers** so each user authenticates as themselves.
- **`headersHelper`** to generate credentials at connection time.

Validate the configuration

To confirm the file is in effect, run two checks on a managed machine:

1. `claude mcp list` shows only the servers in `managed-mcp.json`. If a user's own servers still appear, the file isn't being read; check the path and permissions.
2. `claude mcp add --transport http test https://example.com/mcp` fails with `Cannot add MCP server: enterprise MCP configuration is active and has exclusive control over MCP servers`. The URL doesn't need to be a real server, since the policy check rejects the command before anything is contacted.

Disable MCP entirely

Deploy a `managed-mcp.json` containing an empty server map to block every MCP server:

```
{
  "mcpServers": {}
}
```

Users see no MCP servers in `/mcp`, and `claude mcp add` fails with the enterprise-policy error above. Servers users had previously configured stop loading the next time they start a session, with no warning that policy is the reason.

Allow claude.ai connectors alongside the managed set

Deploying `managed-mcp.json` suppresses **claude.ai connectors** by default, including connectors an administrator configured for the organization in the claude.ai admin console. To load those connectors alongside the servers in `managed-mcp.json`, set `"allowAllClaudeAiMcps": true` in a **managed settings source**. Requires Claude Code v2.1.149 or later.

With the setting enabled, Claude Code loads the same claude.ai connectors it would load if `managed-mcp.json` were not deployed. **Allowlists and denylists** still apply to those connectors, so you can block specific ones with `deniedMcpServers`. The setting affects only claude.ai connectors; plugin-provided servers stay suppressed.

Claude Code reads this setting only from admin-controlled policy tiers: server-managed settings, an MDM-deployed plist or HKLM registry key, or a system `managed-settings.json` file. Placing it in user or project settings has no effect, so users cannot re-enable connectors that exclusive control suppressed.

Policy-based control with allowlists and denylists

Allowlists and denylists filter which configured servers are allowed to load. They aren't a registry: a server still has to be added by a user, a plugin, or `managed-mcp.json` before the allowlist or denylist applies to it. To deploy servers to users, use `managed-mcp.json`.

To make the allowlist authoritative, set `allowedMcpServers` and `allowManagedMcpServersOnly: true` together in a **managed settings source**, such as server-managed settings or a deployed `managed-settings.json` file. **Restrict the allowlist to managed settings only** shows the configuration. Without `allowManagedMcpServersOnly`, allowlists from every settings source merge, including a user's own `~/.claude/settings.json`, so a user can broaden what your allowlist permits. Denylists merge from every source regardless.

> `allowManagedMcpServersOnly` is separate from `allowManagedPermissionRulesOnly`, which locks down **permission rules** only. Setting that flag does not enforce the MCP allowlist.

Match servers by URL, command, or name

`allowedMcpServers` and `deniedMcpServers` are lists of entries. Each entry is an object with a single key that identifies servers by their URL, their command, or their name:

Key	Matches	Use for
`serverUrl`	A remote server URL, exact or with `*` wildcards	HTTP and SSE servers
`serverCommand`	The exact command and arguments that start a stdio server	Stdio servers
`serverName`	The user-assigned label. Exact match only; wildcards are not expanded	Either type, but see the Warning below

Leaving `allowedMcpServers` unset is different from setting it to an empty array:

Setting	Unset (default)	Empty array `[]`	Populated
`allowedMcpServers`	All servers allowed	No servers allowed	Only matching servers allowed
`deniedMcpServers`	No servers blocked	No servers blocked	Matching servers blocked

⚠ A `serverName` entry, in either list, is not a security control. The name is the label a user assigns when running `claude mcp add` or editing a config file, not the underlying server, so a user can call any server `github`. For claude.ai connectors the name is the display name returned by claude.ai, which can change. To enforce which servers actually run, add `serverCommand` or `serverUrl` entries.

The `serverName` validation differs between the two lists:

- In `deniedMcpServers`, `serverName` accepts any non-empty string, so you can block **claude.ai connectors** by their display name. For example, `{ "serverName": "claude.ai Slack" }` blocks the Slack connector. Prefer a `serverUrl` entry when you need the deny to be robust to renames, or when a connector name collides and gains a `(N)` suffix.
- In `allowedMcpServers`, `serverName` is limited to letters, numbers, hyphens, and underscores. Use `serverUrl` to allowlist a claude.ai connector.

To turn off all claude.ai connectors, see `disableClaudeAiConnectors`.

How a server is evaluated

Before loading a server, including one from `managed-mcp.json`, Claude Code runs three checks in order:

1. **Merge the lists.** Allowlist and denylist entries from every settings source combine into one allowlist and one denylist. When `allowManagedMcpServersOnly` is `true`, only the managed allowlist is kept; the denylist always merges from every source.
2. **Check the denylist.** A server that matches any denylist entry, by URL, command, or name, is blocked. Nothing overrides a denylist match.
3. **Check the allowlist.** If `allowedMcpServers` isn't set anywhere, every server that passed the denylist loads. If it is set, what the server must match depends on its type, shown in the table below.

Server type	Allowed when it matches
Remote (HTTP or SSE)	A `serverUrl` entry. A `serverName` match counts only when the allowlist contains no `serverUrl` entries
Stdio	A `serverCommand` entry. A `serverName` match counts only when the allowlist contains no `serverCommand` entries

Two matching rules apply inside those checks:

- **Commands match exactly.** Every argument, in order. `["npx", "-y", "server"]` does not match `["npx", "server"]` or `["npx", "-y", "server", "--flag"]`.
- **URLs support `*` wildcards** anywhere in the pattern, including the scheme. Hostname matching is case-insensitive and ignores a trailing FQDN dot, so `https://Mcp.Example.com/*` matches `https://mcp.example.com/api`. Paths stay case-sensitive.

Pattern	Allows
`https://mcp.example.com/*`	All paths on a specific domain
`https://mcp.example.com`	Also all paths on that domain. A pattern with no path matches any path
`https://*.example.com/*`	Any subdomain of `example.com`
`http://localhost:*/*`	Any port on localhost
`*://mcp.example.com/*`	Any scheme to a specific domain

Example configuration

The configuration below sets up a hard allowlist with a denylist. The highlighted lines change how the rest of the list is evaluated, and the callouts after the block explain each one:

```
{
  "allowedMcpServers": [
    { "serverUrl": "https://api.githubcopilot.com/*" },
    { "serverUrl": "https://mcp.sentry.dev/*" },
    { "serverCommand": ["npx", "-y", "@modelcontextprotocol/server-filesystem", "."]
},
    { "serverCommand": ["python", "/usr/local/bin/approved-
server.py"] },
    { "serverUrl": "https://mcp.example.com/*" },
    { "serverUrl": "https://*.internal.example.com/*" }
  ],
  "deniedMcpServers": [
    { "serverName": "dangerous-server" },
    { "serverCommand": ["npx", "-y", "unapproved-package"] },
    { "serverUrl": "https://*.untrusted.example.com/*" }
  ]
}
```

- **Line 3**: the first `serverUrl` entry. Once one exists, every remote server must match a URL

pattern, so a user can't get an unlisted remote server through by giving it an allowed name.

- **Line 5**: the first `serverCommand` entry. Same effect for stdio servers, so every local server must match a listed command exactly.
- **Line 11**: a `serverName` entry in the denylist. Denylist entries always apply, so any server named `dangerous-server` is blocked regardless of its URL or command.

A `serverName` entry in this allowlist would never match anything, since both transport types already have stricter entries.

The accordions below walk through how a server is evaluated against other allowlist and denylist combinations.

<details>
<summary>URL-only allowlist</summary>
</details>

<details>
<summary>Command-only allowlist</summary>
</details>

<details>
<summary>Mixed name and command allowlist</summary>
</details>

<details>
<summary>Name-only allowlist</summary>
</details>

<details>
<summary>Allowlist with denylist override</summary>
</details>

Restrict the allowlist to managed settings only

To make the managed allowlist the only one that applies, set `allowManagedMcpServersOnly` in the managed settings file:

```
{
  "allowManagedMcpServersOnly": true,
  "allowedMcpServers": [
    { "serverUrl": "https://api.githubcopilot.com/*" },
    { "serverUrl": "https://*.internal.example.com/*" }
  ]
}
```

When `allowManagedMcpServersOnly` is `true`, allowlists from user, project, and local settings are ignored. The denylist still merges from all sources, so users can always block servers for themselves.

How restrictions appear to users

When a restriction blocks a server, the user either sees an error from `claude mcp add` or the server silently stops loading. Use this table to recognize those reports and to tell users what to expect before you roll out a change:

Restriction	What the user sees
`managed-mcp.json` is present and the user runs `claude mcp add`	`Cannot add MCP server: enterprise MCP configuration is active and has exclusive control over MCP servers`
The server is on a denylist and the user runs `claude mcp add`	`Cannot add MCP server "<name>": server is explicitly blocked by enterprise policy`
The server isn't on the allowlist and the user runs `claude mcp add`	`Cannot add MCP server "<name>": not allowed by enterprise policy`
A previously configured server is now blocked by policy	The server silently disappears from `/mcp` and `claude mcp list` with no warning

In the last case, the user gets no signal that policy is the reason their server disappeared, so tell affected users which servers are blocked when you roll out a new restriction.

Monitor MCP usage

When **OpenTelemetry export** is configured, Claude Code can record which MCP servers and tools users invoke. Set `OTEL_LOG_TOOL_DETAILS=1` to include MCP server and tool names in tool events, then aggregate them in your collector to see which servers your users actually connect to. See **Monitoring** to set up the exporter and for the full event schema.

Configuration summary

Every file and setting this page covers, what it controls, and how to deliver it:

Surface	What it controls	Where it lives	How to deliver
`managed-mcp.json`	Fixed server set, exclusive control	System path: `/Library/Application Support/ClaudeCode/`,	MDM, GPO, fleet management, or any process with administrator privileges.

Surface	What it controls	Where it lives	How to deliver
		`/etc/claude-code/`, or `C:\Program Files\ClaudeCode\`	Cannot be set through server-managed settings
`allowedMcpServers`	Allowlist of permitted servers	Any **settings file**; entries from every source merge unless `allowManagedMcpServersOnly` is set	For enforcement, a **managed settings source**: server-managed settings, `managed-settings.json`, MDM profile, or registry
`deniedMcpServers`	Denylist of blocked servers	Any settings file; entries from every source merge	Same as `allowedMcpServers`
`allowManagedMcpServersOnly`	Locks the allowlist to managed sources only	Managed settings sources only; the setting has no effect elsewhere	Same as `allowedMcpServers`
`allowAllClaudeAiMcps`	Loads claude.ai connectors alongside `managed-mcp.json` instead of suppressing them	Managed settings sources only; the setting has no effect elsewhere	Same as `allowedMcpServers`

Related resources

- **Decide what to enforce**: MCP restrictions alongside permission rules, sandboxing, and the other admin controls
- **Connect Claude Code to tools via MCP**: the full MCP reference, including transports, scopes, and authentication
- **Settings**: the settings hierarchy and how managed settings take precedence
- **Server-managed settings**: deliver `allowedMcpServers` and `deniedMcpServers` from the Claude.ai admin console
- **Security**: the threat model these controls defend against
- **Claude Enterprise Administrator Guide**: SSO, SCIM, seat management, and rollout playbook